

# Camada de Transporte

Princípios, Multiplexação, UDP e Introdução ao TCP

Profa. Anelise Munaretto e Prof. Mauro Fonseca

DAINF - UTFPR

## Material de Referência Bibliográfica

Kurose, J. F. & Ross, K. W. — *Redes de Computadores e a Internet: Uma Abordagem Top-Down*

[https://gaia.cs.umass.edu/kurose\\_ross/](https://gaia.cs.umass.edu/kurose_ross/)

O Que a Camada de Transporte Oferece: Serviços e Princípios

Como Organizar os Dados: Multiplexação e Demultiplexação

Como Transferir Sem Conexão: O Protocolo UDP

Como Garantir a Entrega: Transferência Confiável (rdt)

Como Ganhar Velocidade: Janelas Deslizantes (GBN e SR)

# **O Que a Camada de Transporte Oferece: Serviços e Princípios**

---

# Serviços da Camada de Transporte

## 1. Comunicação Lógica Entre Processos

A camada de transporte conecta **aplicações** (processos), enquanto a camada de rede conecta **hosts** (máquinas).

- **Lado Emissor:** Quebra mensagens da aplicação em **segmentos** e os repassa à camada de rede.
- **Lado Receptor:** Reordena e remonta os segmentos, entregando-os ao socket/porta correto.

### Canal Lógico Fim-a-Fim vs. Roteamento Físico Salto-a-Salto



# Analogia Fundamental: Camada de Transporte vs. Rede

## O Cenário das Casas (Analogia de Kurose & Ross)

**Cenário:** 12 crianças da **Casa A** trocam correspondências com 12 crianças da **Casa B**.

- **Crianças:** Processos de aplicação em execução.
- **Cartas:** Mensagens geradas pelas aplicações.
- **Casas:** *Hosts* ou sistemas finais (endereços IP).

## Camada de Transporte

**Ana e Pedro** (os irmãos mais velhos):

- Coletam as cartas dos irmãos e repassam ao carteiro (multiplexação).
- Recebem as cartas do carteiro e entregam em mãos a cada irmão (demultiplexação).
- **Escopo:** Processo-a-Processo.

## Camada de Rede

**Os Correios** (o serviço postal):

- Transportam os lotes de cartas da residência A até a residência B.
- Enxergam apenas o endereço da casa (IP), sem inspecionar os nomes internos.
- **Escopo:** Host-a-Host.

## Princípio Arquitetural de Adição de Valor

O transporte é delimitado pelos limites da rede (não cria vazão infinita), mas pode adicionar **confiabilidade** e garantias de entrega mesmo sobre um canal não confiável.

# TCP vs. UDP: Os Dois Pilares da Camada de Transporte

## 1. TCP (Transmission Control Protocol)

Orientado a Conexão e Confiável:

- **Confiabilidade Estrita:** Retransmissão de perdas e detecção de erros via ACKs/Timeouts.
- **Entrega em Ordem:** Numeração de sequência para remontagem correta do fluxo.
- **Controle de Fluxo e Congestionamento:** Adapta a taxa à janela do receptor e à capacidade da rede.
- **Aplicações:** Web (HTTP), E-mail (SMTP/IMAP), SSH, Transferência de arquivos (FTP).

## 2. UDP (User Datagram Protocol)

Não Conectado e Sem Garantias:

- **Sem Handshake:** Envio imediato no primeiro datagrama gerado (latência mínima de início).
- **Sem Retransmissão:** Não interrompe o fluxo contínuo para esperar pacotes atrasados.
- **Sem Controle de Taxa:** Envia na velocidade requisitada pelo processo de aplicação.
- **Aplicações:** Streaming em tempo real, Telefonia VoIP, DNS, Jogos multiplayer.

## Camada de Rede Subjacente (IP) vs. Camada 4

O protocolo IP opera sob o modelo de **Melhor Esforço (Best Effort)**: o **TCP** constrói confiabilidade e controle sobre uma rede não confiável, enquanto o **UDP** atua como uma interface direta aos datagramas IP com multiplexação por portas.

## **Como Organizar os Dados: Multiplexação e Demultiplexação**

---

# Multiplexação e Demultiplexação: O Papel das Portas

## 1. Multiplexação (Emissor)

Coleta dados de múltiplos *sockets*, encapsula-os com cabeçalhos contendo **portas de origem e destino** e repassa os segmentos à rede.

## 2. Demultiplexação (Receptor)

Examina os campos de porta no cabeçalho do segmento recebido e entrega os dados diretamente ao *socket* da aplicação correta.

## Faixas de Portas (Campos de 16 bits)

- **0 a 1023 (Well-Known):** Serviços padrão de sistema (ex.: *HTTP 80*, *FTP 21*, *DNS 53*).
- **1024 a 49151 (Registered):** Registradas na IANA para aplicações específicas.
- **49152 a 65535 (Dynamic/Ephemeral):** Alocadas temporariamente pelo cliente.

Fluxo Fim-a-Fim: Mux (Cliente) → Rede → Demux (Servidor)



# Como Funciona a Demultiplexação?

## 1. Recepção no Host

O host recebe datagramas IP contendo os endereços **IP de Origem** e **IP de Destino**.

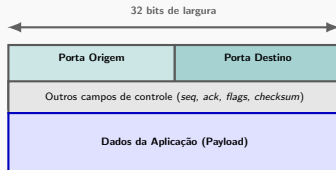
## 2. Extração de Cabeçalho

O segmento transportado revela os campos **Porta de Origem** e **Porta de Destino**.

## 3. Encaminhamento ao Socket

O SO cruza o conjunto de identificadores para rotear a carga útil diretamente ao processo de destino.

## Formato Genérico do Segmento (32 bits)



# Demultiplexação Sem Conexão: O Modelo UDP

## 1. Tupla de Identificação UDP

O socket UDP é indexado pelo SO usando uma **tupla de 2 elementos**:

- (IP\_Destino, Porta\_Destino)
- O SO **ignora** o IP e a porta de origem na hora de entregar o datagrama ao socket.

## 2. Efeito Funil (Funneling)

Datagramas com **origens totalmente distintas** convergem para o **mesmo buffer/socket**, cabendo à aplicação ler a tupla do emissor via `recvfrom()`.

## Múltiplos Clientes → Socket Único



## Implicação Prática

Não há reserva de contexto de sessão no SO; o socket permanece sempre disponível para qualquer novo emissor.

# Demultiplexação Orientada a Conexão: O Modelo TCP

## 1. A Quádrupla TCP (4-Tuple)

Diferente do UDP, o socket TCP é indexado pela combinação estrita de **4 identificadores**:

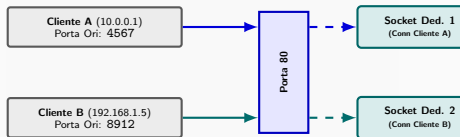
- (IP\_Origem, Porta\_Origem)
- (IP\_Destino, Porta\_Destino)

Todos os 4 valores são avaliados simultaneamente para rotear o segmento ao processo.

## 2. Sockets Dedicados por Sessão

O socket de escuta (*welcome*) permanece na porta de destino, mas cada conexão aceita via `accept()` instancia um **novo socket dedicado** com buffers e números de sequência exclusivos.

## Mapeamento 1:1 por Conexão Ativa



## Servidores Web Concorrentes

Servidores modernos (*Apache, Nginx*) mapeiam cada socket dedicado a uma thread/processo ou laço de eventos assíncrono.

# Comparativo de Demultiplexação: UDP vs. TCP

## Impacto Arquitetural no Projeto de Servidores

O modelo de demultiplexação adotado pelo sistema operacional condiciona o gerenciamento de concorrência, o isolamento de sessões e a alocação de memória na aplicação.

### Matriz Comparativa de Comportamento e Recursos

| Critério             | UDP (Sem Conexão)                                                     | TCP (Orientado a Conexão)                                                                   |
|----------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Indexação do Socket  | <b>Tupla (IP, Porta Destino):</b> descarta a origem no demux.         | <b>Quádrupla Completa:</b> (IP, Porta) <sub>Origem</sub> e (IP, Porta) <sub>Destino</sub> . |
| Isolamento de Fluxos | <b>Ausente:</b> pacotes de múltiplos clientes entram no mesmo buffer. | <b>Estrito:</b> cada conexão lê e grava em um buffer de fluxo isolado.                      |
| Estrutura no Host    | <b>Socket Único:</b> processa requisições sequencialmente por padrão. | <b>Múltiplos Sockets:</b> accept() aloca um socket dedicado por sessão.                     |
| Concorrência         | Complexa: o <i>worker pool</i> deve despachar datagramas manualmente  | Nativa: cada socket dedicado é entregue a uma <i>thread</i> ou <i>event loop</i>            |

# Exemplo Prático TCP: O Servidor com Múltiplas Conexões

## 1. A Demultiplexação pela 4-Tupla

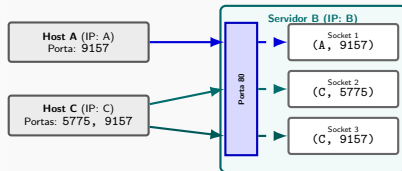
Três fluxos convergem à porta 80 do Servidor B. O SO inspeciona a **quádrupla completa** para mapeá-los:

- (A, 9157, B, 80) → Socket 1
- (C, 5775, B, 80) → Socket 2
- (C, 9157, B, 80) → Socket 3 (mesma porta que A, mas IP distinto)

## 2. Engenharia de Servidores

Servidores de alto desempenho (*Nginx*, *Node.js*) utilizam laços de eventos não bloqueantes via *epoll* / *kqueue* para gerir milhares de conexões ativas simultaneamente sem sobrecarga de memória por *thread*.

## Isolamento de Sockets Dedicados



# Como Transferir Sem Conexão: O Protocolo UDP

---

# UDP: User Datagram Protocol [RFC 768]

## 1. Fundamentos Arquiteturais

Serviço de transporte minimalista (*no-frills*) direto sobre o IP:

- **Sem Conexão:** Sem *handshake* preliminar (latência zero de RTT no início).
- **Melhor Esforço:** Sem retransmissão; segmentos perdidos são descartados.
- **Sem Ordenação:** Datagramas chegam na ordem de trânsito dos roteadores.
- **Sem Estado (Stateless):** Sem tabelas de fluxo ou controle de janela no SO.

## 2. Cenários de Aplicação

Requisitos críticos de baixa latência e simplicidade:

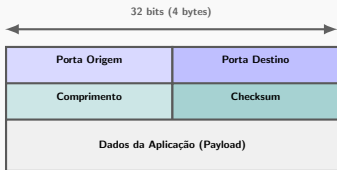
- **Tempo Real:** VoIP, streaming de vídeo ao vivo e partidas multiplayer.
- **Transações Curtas:** DNS e SNMP (um único pacote de ida e volta).
- **Dispositivos Restritos (IoT):** Redes de sensores com memória reduzida e restrição de energia.

## A Revolução Moderna: QUIC e HTTP/3

O UDP tornou-se o substrato da Web moderna: o protocolo **QUIC** opera sobre UDP no **HTTP/3**, implementando criptografia (*TLS 1.3*), controle de perdas e mitigação do bloqueio de cabeça de fila (*Head-of-Line Blocking*) em nível de usuário, sem depender de modificações no *kernel* do sistema operacional.

# Anatomia do UDP: Formato e Motivação

## Cabeçalho Minimalista: 8 Bytes



## Overhead Comparativo

Cabeçalho UDP: **8 bytes** fixos.

Cabeçalho TCP: **20 a 60 bytes** (com campos de opções).

## Por Que Usar o UDP?

- **Latência Mínima de Inicialização:** Sem *handshake* de três vias (RTT zero antes de enviar dados).
- **Sem Estado no Servidor:** Não consome buffers de conexão, temporizadores ou tabelas de sequência na memória.
- **Controle de Taxa Livre:** Sem mecanismo de controle de congestionamento forçado; a aplicação dita o ritmo de transmissão.
- **Cabeçalho Compacto:** Ideal para rajadas frequentes de mensagens curtas (reduz o desperdício de banda).

## Verificação com Checksum

Oferece detecção de erros fim-a-fim simples (cálculo de soma em complemento de 1), mas não realiza recuperação ou descarte inteligente.

# UDP Checksum: Princípios e Funcionamento

## 1. No Lado do Emissor

- Trata o conteúdo (cabeçalho, pseudo-cabeçalho IP e payload) como inteiros de **16 bits**.
- Calcula a soma binária com transporte no bit menos significativo (*wrap-around*).
- Aplica a negação bit a bit (**complemento de 1**) e grava no campo Checksum.

## 2. No Lado do Receptor

- Soma todos os campos de 16 bits juntamente com o checksum recebido.
- **Condição de Sucesso:** O resultado deve ser composto exclusivamente por 1s (0xFFFF).
- **Detecção de Erro:** Se houver qualquer bit 0, o datagrama foi corrompido (descarte ou repasse com alerta).

## O Pseudo-Cabeçalho IP

O cálculo inclui campos da Camada 3 (**IP Origem, IP Destino, Protocolo UDP e Comprimento**) para evitar entregas a hosts ou portas incorretas por erros de roteamento local.

## Limitações Matemáticas

- **Inversões Múltiplas:** Erros compensatórios em posições opostas podem somar zero e mascarar corrupções de bits.
- **Sem Recuperação:** Apenas detecta falhas; o UDP não solicita retransmissão de pacotes corrompidos.

## O Princípio Fim-a-Fim (End-to-End Argument)

Mesmo que enlaces intermediários forneçam checagem de erros (como o CRC da camada de enlace), verificações na camada de transporte são obrigatórias para proteger contra falhas de memória em buffers de roteadores.

# Exemplo Matemático: Checksum da Internet

## 1. Regra de Adição Binária

- Os dados são divididos em palavras de **16 bits**.
- Se a soma das palavras gerar transbordo (*carry-out*) no 17º bit, esse bit excedente deve ser somado no bit menos significativo (*wraparound*).

## 2. Geração do Checksum

Após aplicar todos os *wraps*, inverte-se cada bit do resultado (operação NOT). O valor resultante é inserido no campo **Checksum** do cabeçalho.

## Cálculo Passo a Passo (Dois Inteiros)

|                    |                                |
|--------------------|--------------------------------|
| 1110011001100110   | (Palavra 1)                    |
| + 1101010101010101 | (Palavra 2)                    |
| <hr/>              |                                |
| 1 1011101110111011 | ← Carry gerado (17º bit)       |
| +                  | 1 (Soma de volta / Wraparound) |
| <hr/>              |                                |
| 1011101110111100   | (Soma com wraparound)          |
| <hr/>              |                                |
| 0100010001000011   | (Checksum: Bits invertidos)    |

## Validação no Receptor

Ao receber os dados, o receptor soma as palavras com o checksum:

$(1011101110111100)_2 + (0100010001000011)_2 = (1111111111111111)_2$ . Se todos os 16 bits forem 1 (0xFFFF), o segmento é validado como íntegro.

## **Como Garantir a Entrega: Transferência Confiável (rdt)**

---

# Princípios da Transferência Confiável de Dados (RDT)

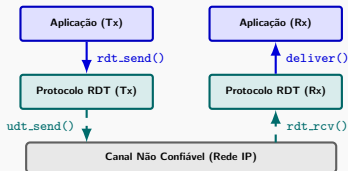
## 1. O Dilema Fundamental

A Camada de Aplicação exige a abstração de um **canal totalmente confiável** (sem perdas, corrupções ou duplicatas), mas a Camada de Rede subjacente opera sob o modelo de **melhor esforço**.

## 2. Primitivas de Serviço

- `rdt_send()`: invocada pela aplicação de envio para transferir dados ao RDT.
- `udt_send()`: chamada pelo RDT para injetar pacotes no canal inseguro.
- `rdt_rcv()`: chegada de pacotes do canal na camada receptora.
- `deliver_data()`: repasse dos dados íntegros à aplicação de destino.

## Modelo de Camadas e Interfaces de API



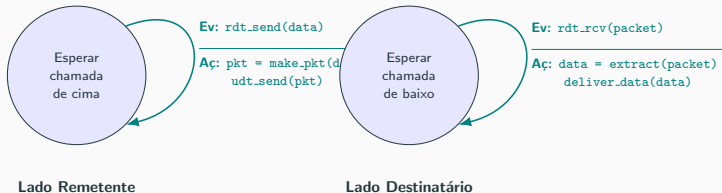
## Objetivo do RDT

Implementar mecanismos de temporização, confirmação (*ACK*) e controle de integridade para isolar completamente a camada superior das falhas da rede física.

# RDT 1.0: O Canal Perfeitamente Confiável

Para iniciar o raciocínio, assumimos um caso utópico: o canal inferior é perfeitamente confiável (não há erros de bits e nenhum pacote é perdido)

- Com um canal perfeito, o protocolo de transporte é trivial. Ele apenas aceita dados da aplicação e os joga na rede
- Modelamos isso usando **Máquinas de Estado Finito (FSM)**. A notação padrão para as transições é: **Evento / Ação**



# RDT 1.0: Transferência sobre Canal Perfeito

## 1. Premissa do Canal

Assume-se um canal subjacente ideal: **zero corrupção de bits** e **zero perda de pacotes**. Não há necessidade de temporizadores, confirmações ou controle de fluxo.

## 2. Notação FSM (Kurose & Ross)

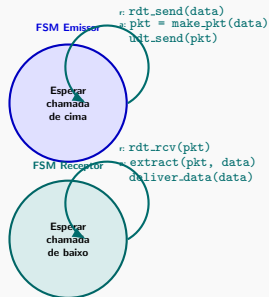
As Máquinas de Estado Finito seguem o padrão:

Evento que desencadeia a transição

Ações executadas pelo protocolo

Se não houver ação a tomar, indica-se por  $\Lambda$ .

## FSMs Uniestado: Emissor e Receptor



## Comportamento do Sistema

Tanto o emissor quanto o receptor possuem apenas **um estado**: o receptor sempre processa os pacotes na velocidade em que chegam, e o emissor injeta dados assim que solicitados pela aplicação.

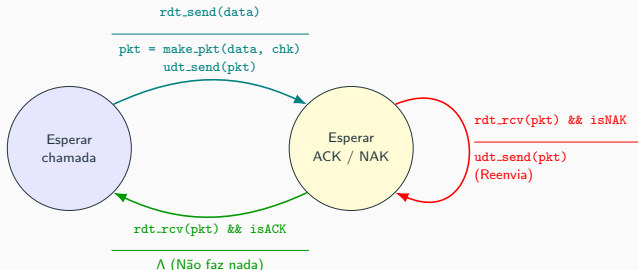
# RDТ 2.0: Canal com Erros de Bits

Redes reais sofrem interferências e corrompem pacotes. Como o remetente sabe se o receptor recebeu a mensagem corretamente?

Precisamos de **ARQ** (*Automatic Repeat reQuest*)

**Novos mecanismos necessários:**

1. **Deteccão de Erros:** O uso do *Checksum*
2. **Feedback do Receptor:** Mensagens de controle indicando o estado
  - **ACK** (*Acknowledge*): "Tudo certo, entendi!"
  - **NAK** (*Negative Acknowledge*): "Houve erro, repita por favor"
3. **Retransmissão:** O remetente reenvia o pacote se receber um NAK



# O Problema Oculto do RDT 2.0

O RDT 2.0 parece sólido, mas possui uma falha fatal (*Flaw*): **E se o pacote de ACK ou NAK também for corrompido durante o trajeto de volta?**

## O Dilema do Remetente

Se o remetente recebe uma resposta incompreensível (corrompida), ele não sabe o que aconteceu no destinatário. A única saída segura é **retransmitir** o pacote

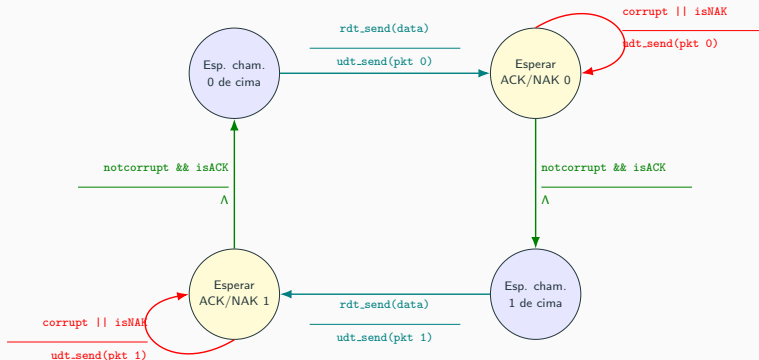
## O Novo Problema: Pacotes Duplicados

- Se o remetente retransmitir após um ACK corrompido, o destinatário receberá a mesma mensagem duas vezes
- O destinatário não terá como saber se o novo pacote é uma retransmissão do pacote anterior ou se já é a próxima mensagem (*nova*) vinda da aplicação!

**A Solução:** O remetente precisa adicionar um **Número de Sequência** ao cabeçalho. Como este é um protocolo “Pare e Espere”, bastam dois números: **0** e **1**

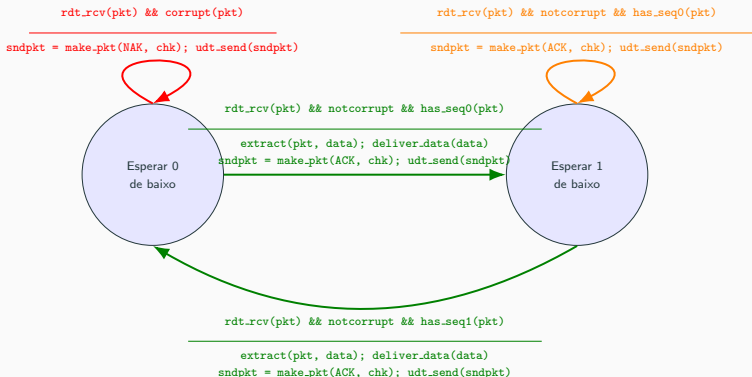
## RDT 2.1: Resolvendo Duplicações (Lado Remetente)

O remetente agora anexa o número de sequência (0 ou 1) ao pacote e a FSM dobra de tamanho para rastrear o estado atual (se está lidando com o pacote 0 ou pacote 1)



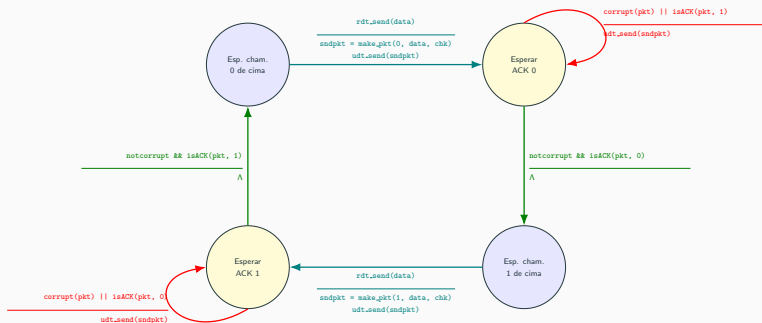
## RDT 2.1: Resolvendo Duplicações (Lado Destinatário)

O destinatário também precisa de dois estados para verificar se o número de sequência recebido é o esperado. Se receber um pacote corrompido, envia NAK; se receber um pacote com sequência trocada (duplicado), apenas reenvia o ACK



## RDT 2.2: Transferência Confiável Sem NAK

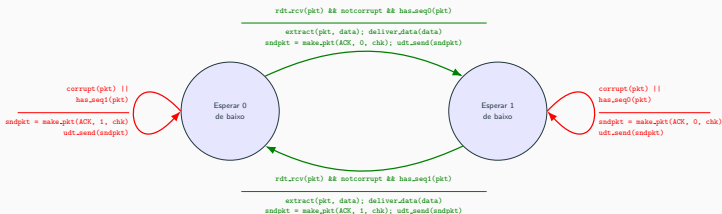
O RDT 2.2 simplifica o protocolo ao remover as mensagens de NAK. O receptor agora deve incluir obrigatoriamente o **número de sequência** no ACK. Um ACK duplicado no remetente tem o mesmo efeito que um NAK: retransmissão do pacote atual



## RDT 2.2: Lado Destinatário (Sem NAK)

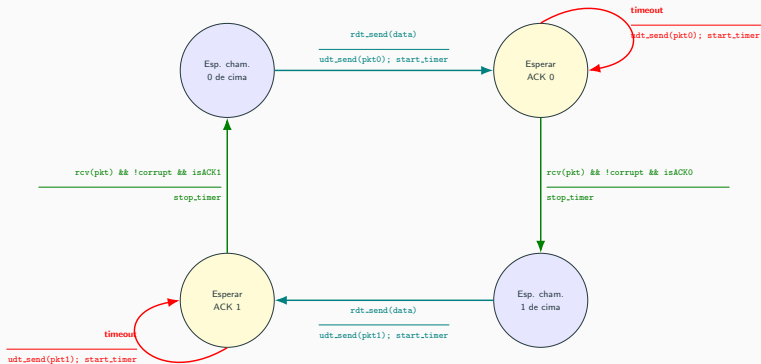
O destinatário no RDT 2.2 não utiliza mais o sinal de NAK. Para sinalizar um erro, ele simplesmente envia um ACK do último pacote recebido com sucesso, incluindo explicitamente o número de sequência

- Se o receptor espera o **pacote 0**, mas recebe algo corrompido ou o **pacote 1**, ele envia um **ACK 1**
- Isso força o remetente a retransmitir, pois ele receberá um “ACK inesperado”



# RDT 3.0: Canais com Erros e Perda de Pacotes

O rdt 3.0 resolve o problema de pacotes perdidos utilizando **temporizadores**. Se um ACK não chega a tempo, o remetente retransmite o pacote automaticamente

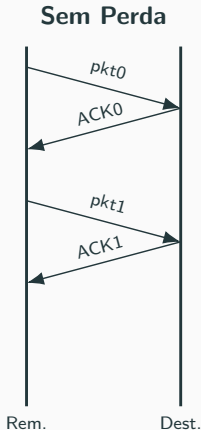


# RDT 3.0 em Operação: Cenários de Perda

O protocolo de bit alternante utiliza o tempo para detectar falhas. Abaixo, comparamos o fluxo normal com o cenário de perda de pacote

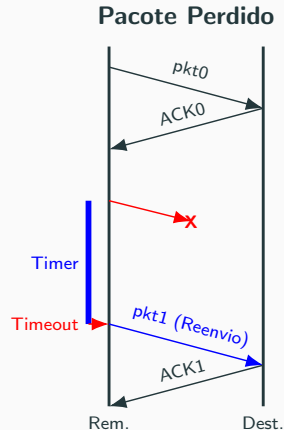
## Operação Sem Perda:

- Fluxo “pare-e-espere” padrão
- O remetente só avança após a confirmação do bit atual



## Perda de Pacote:

- O pacote se perde no canal (X)
- O *timeout* garante a retransmissão sem travamento do sistema

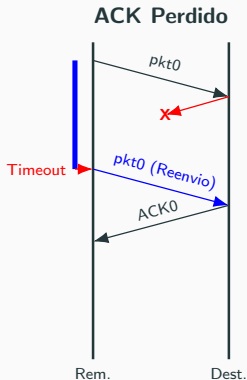


# RDT 3.0 em Operação: ACK Perdido e Atrasado

O protocolo lida com falhas no caminho de volta (do receptor para o remetente) garantindo que duplicatas sejam descartadas corretamente

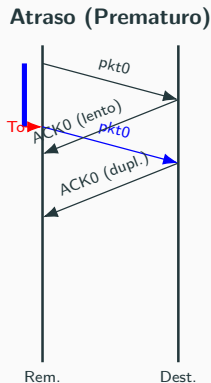
## ACK Perdido:

- O remetente retransmite após o *timeout*
- O receptor detecta o número de sequência duplicado e apenas reenvia o ACK



## Temporização Prematura:

- O *timeout* ocorre antes do ACK chegar
- Gera retransmissão desnecessária e ACKs duplicados no remetente

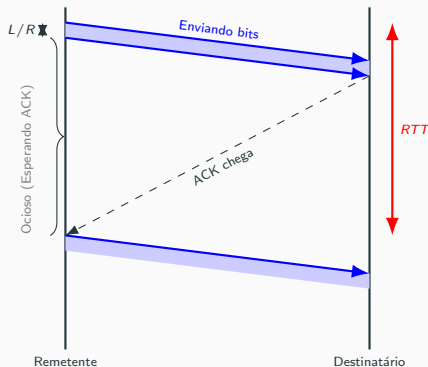


# Desempenho do RDT 3.0: O Problema da Ineficiência

Apesar de ser confiável, o RDT 3.0 sofre com o modelo **Pare-e-Espere**. O remetente fica limitado pela latência da rede ( $RTT$ ), resultando em uma baixíssima utilização do canal

- **Exemplo Teórico:** Link de 1 Gbps,  $RTT$  de 30 ms e pacotes de 1 KB (8000 bits)
- **Tempo de Transmissão ( $d_{trans}$ ):**  $L/R = 8.000/10^9 = 8$  microssegundos
- **Utilização do Remetente ( $U_{sender}$ ):** Fração do tempo em que o remetente está enviando bits

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{0,008 \text{ ms}}{30,008 \text{ ms}} \approx 0,00027$$



## **Como Ganhar Velocidade: Janelas Deslizantes (GBN e SR)**

---

# Pipelining: Aumentando a Utilização do Canal

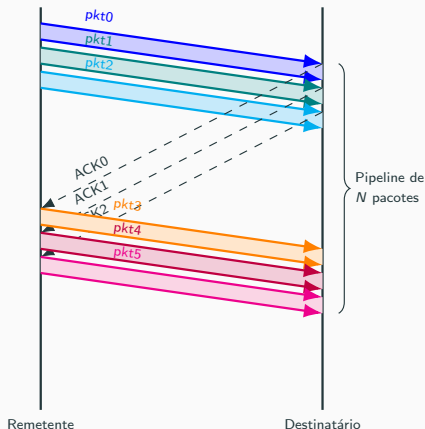
A solução para a ineficiência do “Pare-e-Espere” é o **Pipelining**: o remetente envia múltiplos pacotes sem esperar por um reconhecimento imediato

## Impactos no Protocolo:

- Números de sequência precisam de maior intervalo
- Necessidade de **buffers** no remetente e receptor

**Melhoria na Utilização:** Com  $N$  pacotes em voo, a eficiência cresce linearmente:

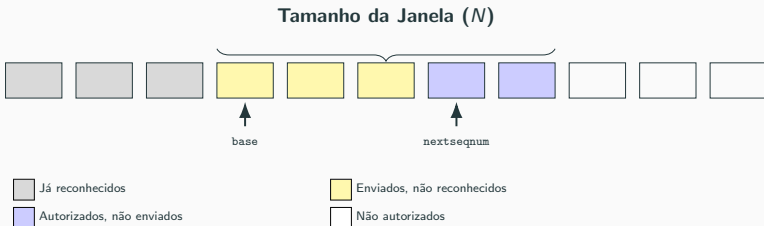
$$U_{sender} = \frac{N \times (L/R)}{RTT + (L/R)}$$



# Go-Back-N (GBN): A Janela do Remetente

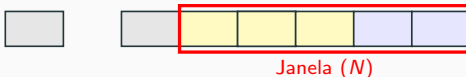
No protocolo GBN, o remetente pode transmitir múltiplos pacotes sem esperar por um reconhecimento imediato. Entretanto, o número de pacotes não reconhecidos no “pipeline” é limitado a um valor máximo  $N$

- **Tamanho da Janela ( $N$ ):** Define a quantidade máxima de pacotes que podem estar em trânsito simultaneamente
- **Deslizamento:** A janela avança (desliza) sempre que o reconhecimento (ACK) do pacote mais antigo na fila (*base*) é recebido



# Go-Back-N (GBN): Lógica do Remetente

O remetente mantém uma janela de tamanho  $N$  e um único temporizador



## 1. Envio (`rdt_send`):

- Se janela não está cheia:
  - Envia pacote (`nextseq`);
  - Se `base == nextseq`, inicia o **timer**;
  - `nextseq++`
- Senão: recusa os dados

## 2. Timeout:

- Reinicia o **timer**;
- **Reenvia todos** os pacotes da janela (de `base` até `nextseq-1`)

## 3. Recebimento de ACK:

- **ACK( $n$ )**: Reconhecimento cumulativo (todos até  $n$  recebidos)
- Atualiza `base = n + 1`;
- Se `base == nextseq`:
  - Para o **timer** (janela vazia)
- Senão:
  - Reinicia o **timer** para o pacote mais antigo pendente

# Go-Back-N (GBN): Lógica do Destinatário

O destinatário GBN é extremamente simples, pois não precisa de buffers para armazenar pacotes fora de ordem

## Comportamento Principal:

- **ACK Somente-em-Ordem:** Envia um ACK para o pacote com maior número de sequência recebido **em ordem**
- Gera ACKs duplicados se necessário.
- Mantém apenas o `expectedseqnum`

## Recepção de Pacote $n$ :

- Se  $n == \text{expectedseqnum}$ :
  - Aceita o pacote;
  - Envia  $\text{ACK}(n)$ ;
  - $\text{expectedseqnum}++$

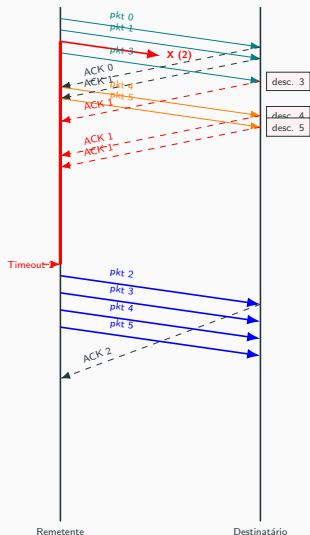
## Pacotes Fora de Ordem:

- **Descarte:** Qualquer pacote  $n \neq \text{expectedseqnum}$  é simplesmente descartado
- **Re-ACK:** O receptor reenvia o ACK do último pacote recebido corretamente (o “topo” da sequência em ordem)



# GBN em Ação: ACKs Duplicados e Retransmissão

O destinatário reenvia o **último ACK em ordem** para cada pacote descartado, sinalizando ao remetente que o fluxo “travou”



## Análise:

- **ACK 1 Repetido:** O receptor avisa 3 vezes que ainda espera o pacote 2
- **Desperdício:** O remetente recebe os ACKs, mas como o GBN não é seletivo, ele ignora e espera o timeout
- **Ação:** No timeout, a janela inteira (2 a 5) volta para a rede

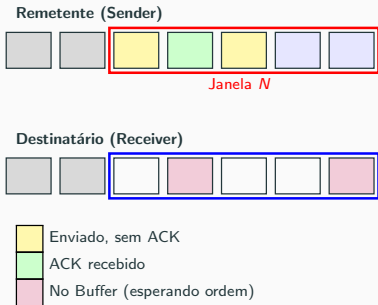
# Selective Repeat (SR): Retransmissão Seletiva

O SR resolve a ineficiência do GBN ao permitir que o receptor armazene pacotes fora de ordem em um **buffer**, evitando o reenvio desnecessário de dados que já chegaram bem

## Mecanismos Chave:

- **ACKs Individuais:** O receptor confirma cada pacote individualmente (não é cumulativo)
- **Buffer no Receptor:** Pacotes fora de ordem são guardados até que as “lacunas” sejam preenchidas
- **Temporizadores Independentes:** O remetente mantém um timer para cada pacote enviado e não reconhecido

## Visão das Janelas (SR)



# Selective Repeat (SR): Lógica de Operação

O SR exige que ambos os lados gerenciem janelas e estados individuais para cada pacote

## Lógica do Remetente:

- **Dados de cima:** Se o seq # estiver na janela, envia o pacote
- **Timeout( $n$ ):** Reenvia apenas o pacote  $n$  e reinicia o timer de  $n$
- **ACK( $n$ )** em  $[base, base + N - 1]$ :
  - Marca pacote  $n$  como recebido;
  - Se  $n$  for o menor pendente ( $base$ ), desliza a janela até o próximo não reconhecido

## Lógica do Destinatário:

- **Pkt  $n$  em  $[rcv\_base, rcv\_base + N - 1]$ :**
  - Envia **ACK( $n$ )**;
  - Se fora de ordem, faz o buffer;
  - Se em ordem, entrega e desliza
- **Pkt  $n$  em  $[rcv\_base - N, rcv\_base - 1]$ :**
  - Envia **ACK( $n$ )** (essencial para o remetente avançar)
- **Caso contrário:** Ignora

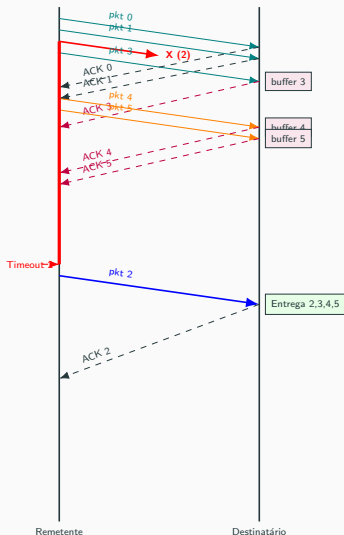
Janela Remetente

Janela Destinatário

*As janelas deslizam de forma independente*

# SR em Ação: Retransmissão Seletiva

Diferente do GBN, o SR evita retransmissões redundantes. O receptor armazena pacotes fora de ordem e confirma cada um individualmente



## Análise SR:

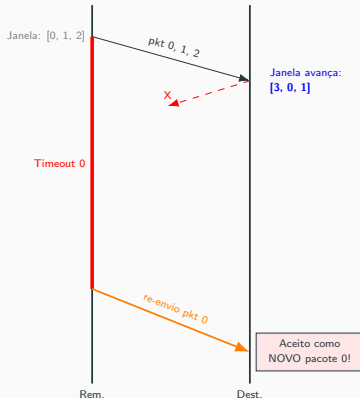
- **ACKs Seletivos:** O remetente sabe exatamente que 3, 4 e 5 chegaram
- **Retransmissão Única:** No timeout, apenas o pkt 2 volta para a rede
- **Buffer:** O receptor segura os dados até que a “falha” (pkt 2) seja corrigida

# Selective Repeat: O Dilema da Janela

Para evitar ambiguidades, o tamanho da janela  $N$  deve ser  $\leq$  à metade do espaço de endereçamento. Caso contrário, o receptor não distingue pacotes novos de retransmissões

## Cenário de Erro ( $N = 3$ , Seq 0-3):

1. Remetente envia **pkt0**, **pkt1**, **pkt2**
2. Receptor recebe os três, envia ACKs e **move a janela** para frente
3. A nova janela do receptor espera por: **[3, 0, 1]**
4. Todos os ACKs são perdidos no canal
5. **Timeout** no remetente para o **pkt0**
6. Remetente retransmite **pkt0** (o “velho”)
7. **O Desastre:** O receptor vê “0”, olha para sua janela [3, 0, 1] e aceita o dado como novo



# Comparativo: Go-Back-N (GBN) vs. Selective Repeat (SR)

A escolha entre GBN e SR depende do equilíbrio entre a simplicidade de implementação e a eficiência no uso da largura de banda

| Característica | Go-Back-N (GBN)                    | Selective Repeat (SR)         |
|----------------|------------------------------------|-------------------------------|
| Reconhecimento | Cumulativo (ACK $n$ )              | Individual por pacote         |
| Temporizador   | Único (para o pacote <i>base</i> ) | Individual (para cada pacote) |
| Buffer (Dest.) | Nenhum (descarta fora de ordem)    | Sim (armazena fora de ordem)  |
| Retransmissão  | Janela completa (agressiva)        | Apenas o pacote perdido       |
| Complexidade   | Baixa (fácil implementação)        | Alta (maior uso de memória)   |

- **GBN:** Ideal para links com baixa taxa de erro onde o descarte não é frequente
- **SR:** Ideal para links ruidosos onde retransmitir a janela inteira seria um desperdício

# Resumo: A Evolução dos Protocolos rdt

A jornada do rdt 1.0 ao 3.0 reflete a construção de mecanismos para lidar com a imperfeição dos canais físicos

| Versão    | Problema Resolvido        | Novos Mecanismos        |
|-----------|---------------------------|-------------------------|
| 1.0       | Nenhum (Canal Ideal)      | Envio/recepção simples  |
| 2.0       | Erros de Bit              | ACK/NAK                 |
| 2.1       | ACK/NAK Corrompidos       | Números de Seq (0 e 1)  |
| 2.2       | Eliminação de NAKs        | ACKs duplicados         |
| 3.0       | Perda de Pacotes/ACKs     | Temporizadores (Timers) |
| Pipelined | Baixa Utilização do Canal | Janelas (GBN e SR)      |

## Resumo dos Mecanismos:

- **Checksum:** Detecta erros de bit
- **Seq #:** Lida com duplicatas e ordem
- **Timer:** Lida com perdas e atrasos
- **Janela:** Permite múltiplos pacotes em trânsito

